Hazmat Classifier

Usage Guide

Ensemble ML + LLM Fallback Pipeline
for Mercado Libre Product Classification

Author: Wesley Silva Alves

Date: March 2026

FastAPI (:8080) | MLflow (:5000) | Gradio Dashboard (:7860)

1. Installation and Setup

1.1 Prerequisites

- Python >= 3.11
- uv package manager (https://docs.astral.sh/uv/)
- libomp (macOS only, for XGBoost): brew install libomp
- Gemini API key (Google AI Studio)
- Mercado Libre API credentials (for data collection)
- Docker (optional, for containerized deployment)

1.2 Install with uv (recommended)

```
# Clone the repository
git clone https://github.com/Wesley-S-Alves/Hazmat.git
cd Hazmat

# Install all dependencies
make install
# or: uv sync --frozen

# Configure environment
cp .env.example .env
# Edit .env with your GEMINI_API_KEY
```

1.3 Install without uv

```
# Create virtual environment
python -m venv .venv
source .venv/bin/activate

# Install dependencies from pyproject.toml
pip install -e .
pip install -e '.[test,lint,dev]'
```

1.4 Docker

```
# Build image
make docker-build
# or: docker build -t hazmat-classifier .

# Run with docker-compose (mounts models/ volume)
make docker-run
# or: docker compose up

# Models are NOT baked into the image.
# They are mounted at runtime via volume: ./models:/app/models
# Set GEMINI_API_KEY in .env or pass via environment.
```

2. Starting Services

2.1 Start Everything (recommended)

```
make serve
```

This single command starts all three services:

Service	Port	Description
FastAPI	8080	REST API for classification, health, metrics, drift, canary
MLflow UI	5000	Experiment tracking, model registry, prompt registry
Gradio	7860	Interactive dashboard consuming the FastAPI backend

2.2 Start Individual Services

```
# API only (with hot reload for development)
make api

# Dashboard only (requires API running)
make dashboard

# MLflow UI only
make mlflow
```

3. Gradio Dashboard (localhost:7860)

The dashboard consumes the FastAPI backend and provides a visual interface for classification, batch processing, and health monitoring.

3.1 Single Item Classification

Enter a product title, optional description, and category ID. Click Classify to see the result with confidence score, source layer, and reason. Pre-loaded examples are available at the bottom of the page.

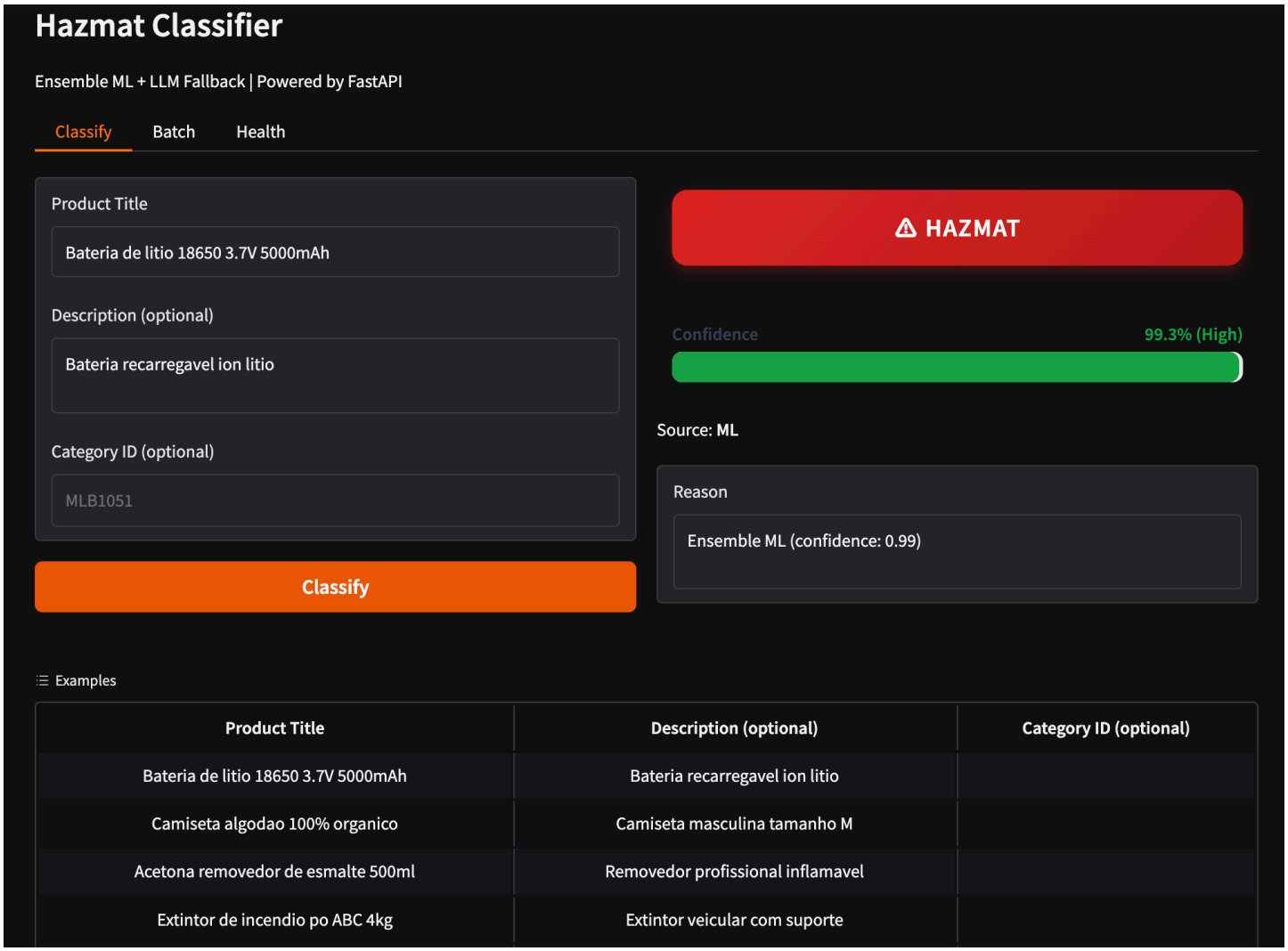

Fig 1: Hazmat classification - Bateria de litio 18650 (99.3% confidence)

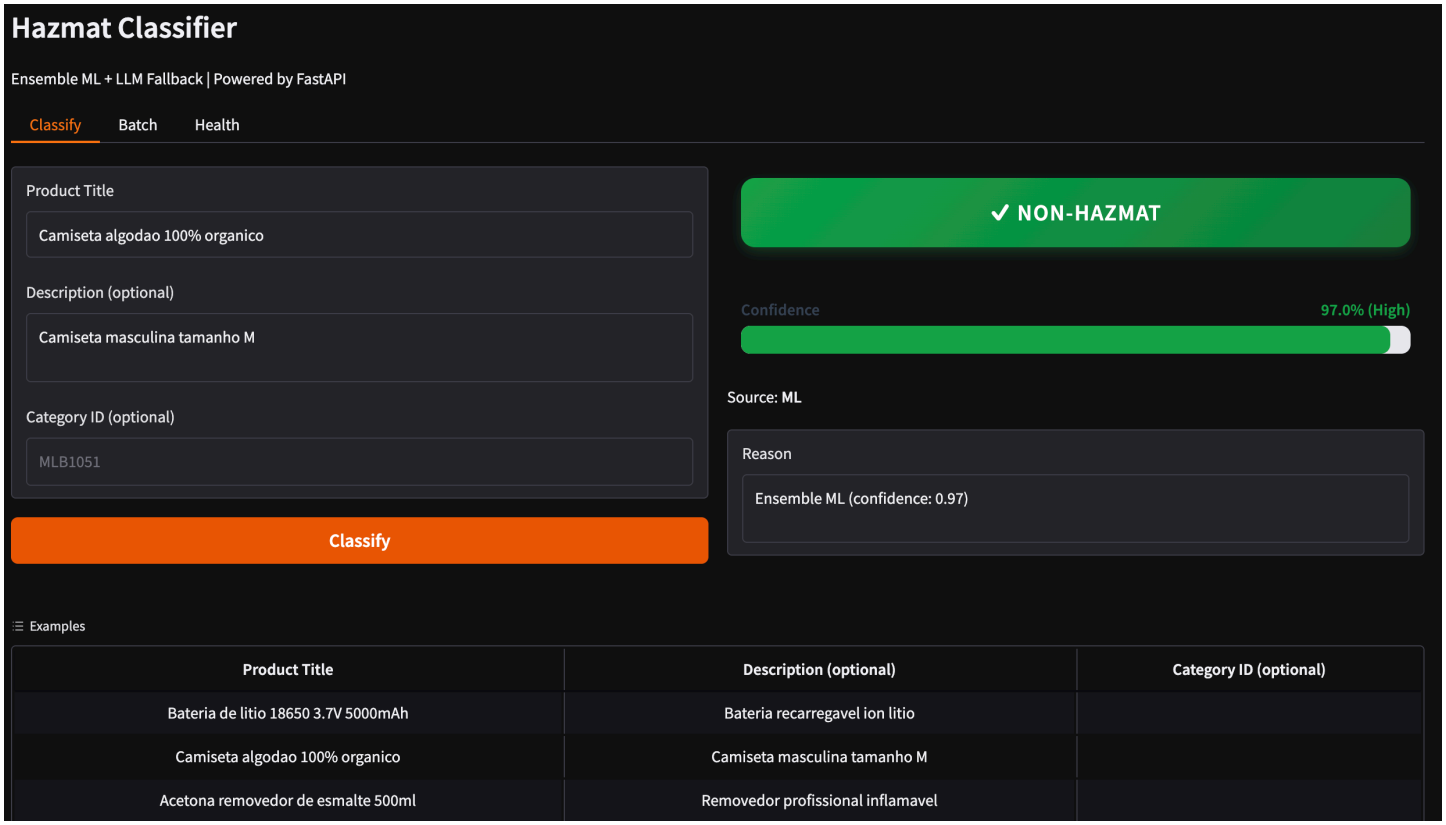

Fig 2: Non-hazmat classification - Camiseta algodao (97.0% confidence)

3.2 Batch Classification

Upload a CSV file with a 'title' column (and optional 'description', 'category_id' columns), or paste multiple product titles (one per line). The system classifies up to 500 items and returns a results table with summary statistics.

3.3 Health Monitoring

The Health tab shows API status, device info (cpu/apple-silicon), model version, and uptime. Links to MLflow UI for full experiment tracking and metrics.

4. REST API (localhost:8080)

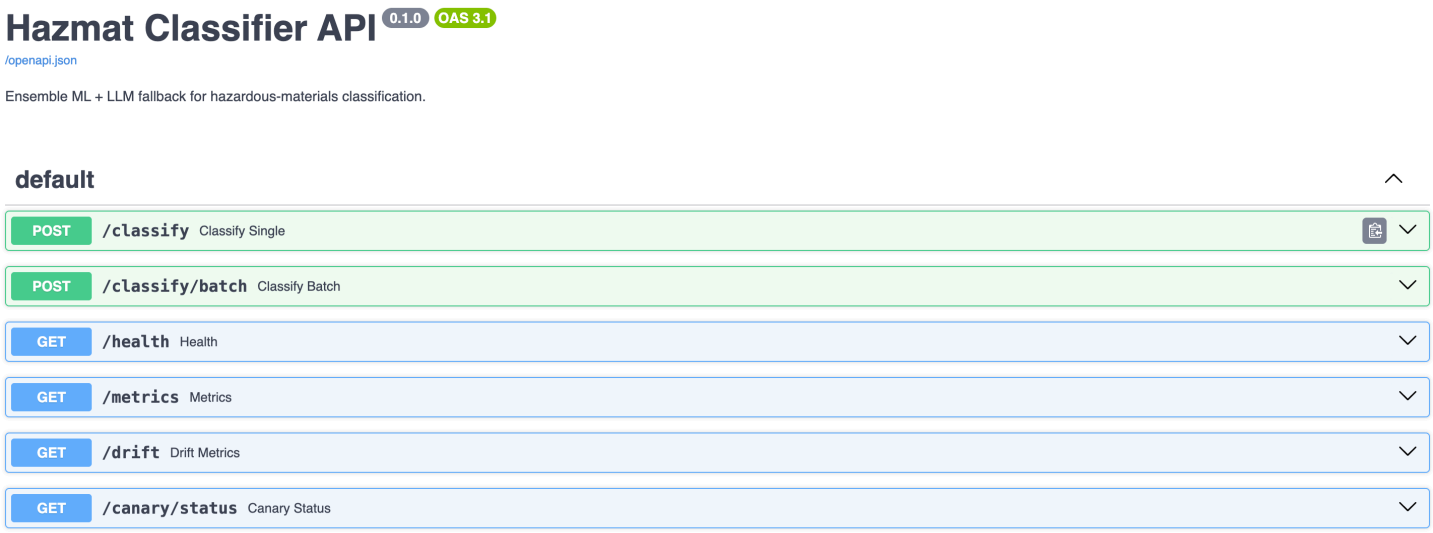

Fig 3: FastAPI Swagger UI - 6 endpoints

4.1 POST /classify - Single Item

```
curl -X POST http://localhost:8080/classify \
  -H "Content-Type: application/json" \
  -d '{"title": "Bateria litio 18650 3.7V",
       "description": "Bateria recarregavel",
       "category_id": "MLB1051"}'
```

Response includes: is_hazmat (bool), confidence_score (0-1), source_layer (ml/llm), reason.

4.2 POST /classify/batch - Batch (up to 100 items)

```
curl -X POST http://localhost:8080/classify/batch \
  -H "Content-Type: application/json" \
  -d '{"items": [
    {"title": "Gasolina 5L"},
    {"title": "Camiseta algodao"}
  ]}'
```

Returns per-item results + summary with total, hazmat count, ML/LLM breakdown.

4.3 GET /health

```
curl http://localhost:8080/health
```

Returns status (ok/degraded), models_loaded, model_version, uptime, device.

4.4 GET /metrics - Prometheus Format

```
curl http://localhost:8080/metrics
```

Exports counters, gauges, and histograms in Prometheus exposition format. Includes classifications per layer, latency percentiles, confidence distribution, hazmat rate, error counts, and drift PSI scores.

4.5 GET /drift - Drift Detection

```
curl http://localhost:8080/drift
```

Returns PSI-based drift metrics comparing a rolling window of predictions against the training baseline. Reports prediction drift, confidence drift, feature-space drift, and active alerts.

4.6 GET /canary/status - Canary Deployment

```
curl http://localhost:8080/canary/status
```

Returns canary routing state: active model versions, traffic split, per-version metrics (latency, confidence, error rate), and auto-evaluation decisions.

5. ML Engineering Workflow

5.1 MLflow Experiment Tracking

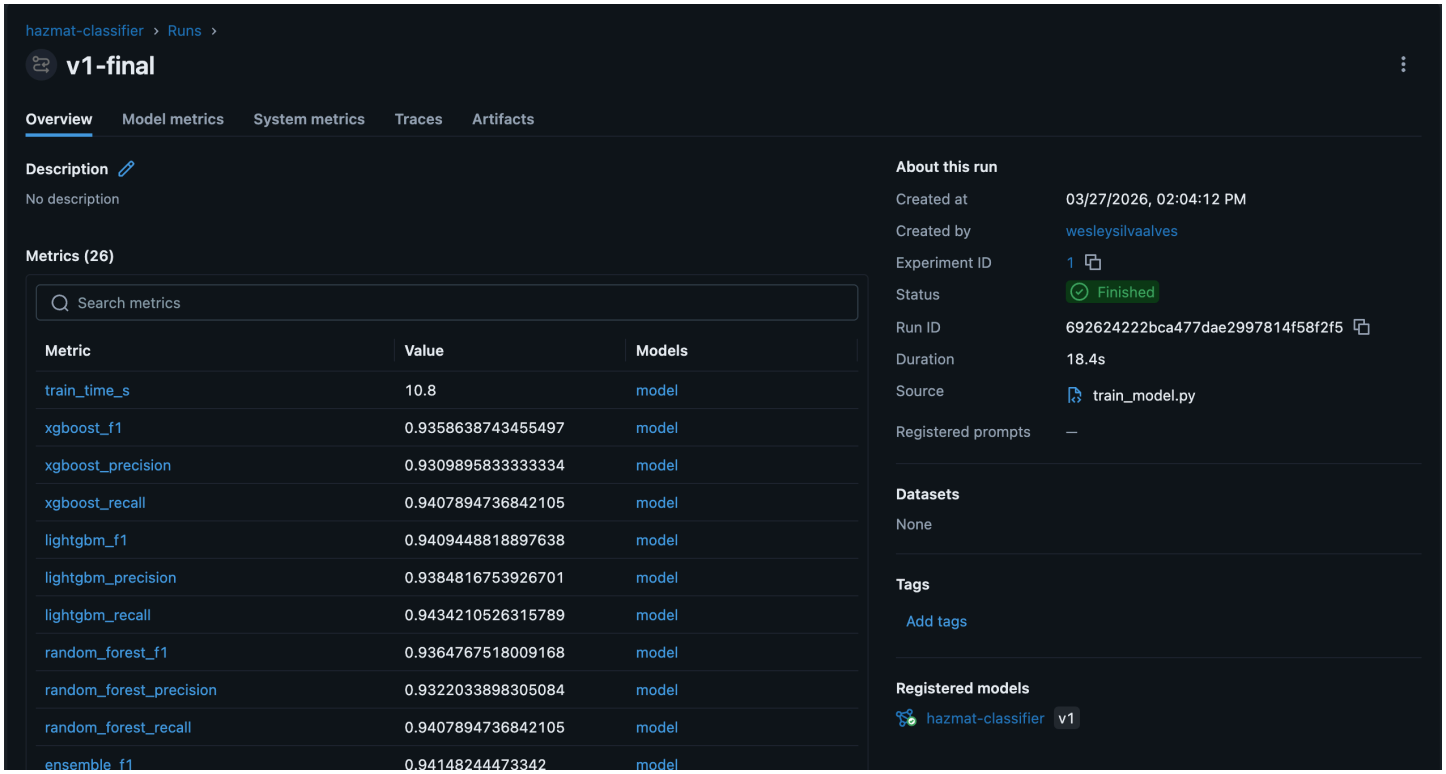

Fig 4: MLflow run detail - 26 metrics, registered model v1

Every training run logs params, metrics, artifacts, and registers the model in MLflow Model Registry. The run tracks: per-model F1/precision/recall, ensemble metrics, ROC AUC, false negative analysis, feature config, decision threshold, and data lineage.

5.2 MLflow Prompt Registry

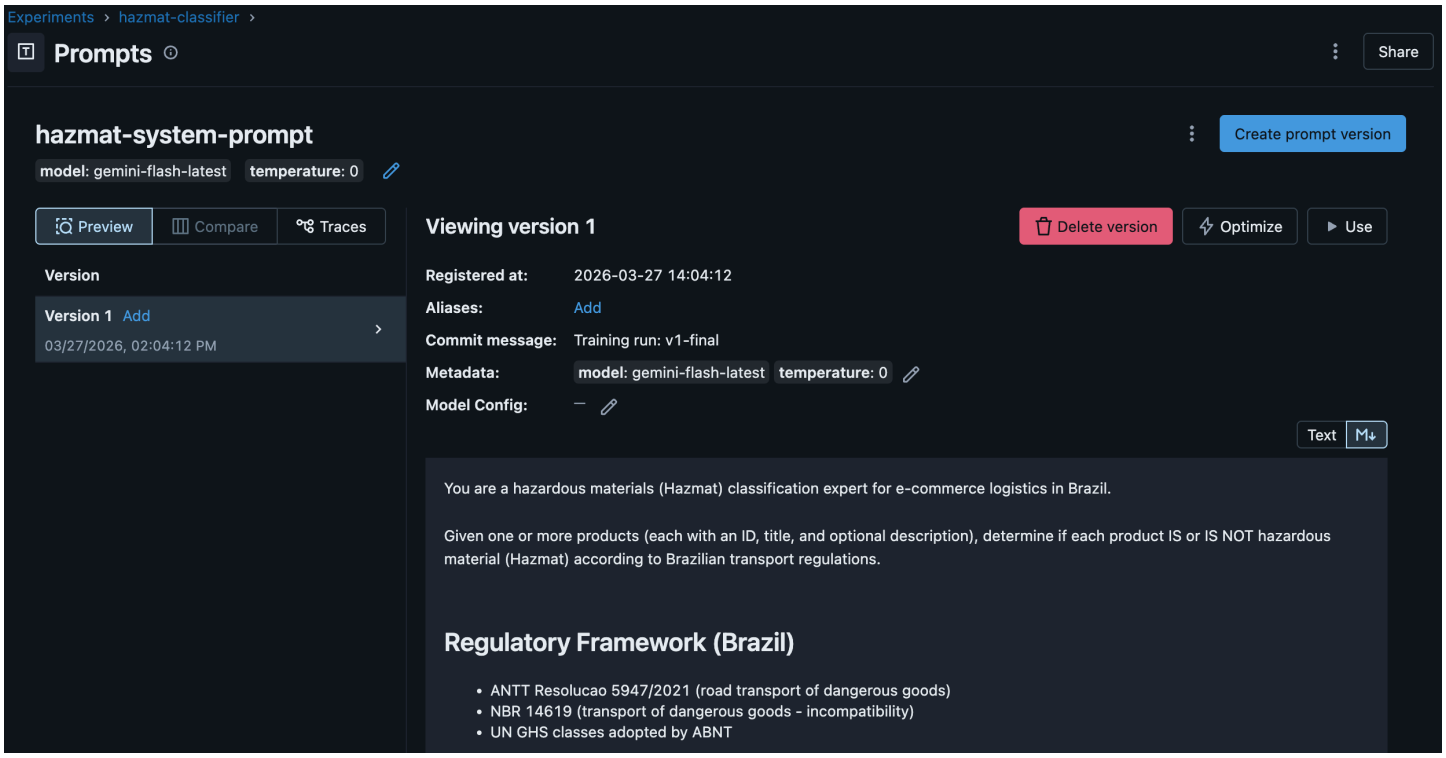

Fig 5: MLflow Prompt Registry - versioned LLM system prompt

The LLM system prompt is versioned in MLflow Prompt Registry. Each prompt change creates a new version with commit message, tags (model, temperature), and audit trail. This enables prompt rollback if a new version degrades quality.

5.3 Model Promotion Workflow

```
# Train and register a new model version
make train

# Validate against golden set (blocks if F1 < 0.85 or FN > 8%)
make validate

# Promote to staging (auto-validates first)
make promote-staging

# Promote to production (auto-validates first)
make promote-production

# List all model versions with stage and metrics
make models-list
```

Promotion follows a gated None -> Staging -> Production workflow. Each promotion automatically runs golden set validation and blocks if the model fails quality gates.

6. Full Pipeline (End-to-End)

Run each step in order to reproduce the entire pipeline from scratch:

Step 1: Collect Data

```
uv run python scripts/collect_data.py
```

Collects ~100k product listings from the Mercado Libre API across 100+ search queries. Saves raw Parquet files and consolidated data.

Step 2: Generate Labels

```
uv run python scripts/generate_labels.py --sample-size 10000
```

Uses Gemini Flash in async multi-item batches (20 items/request, 20 concurrent) to label a stratified sample. Saves incrementally to Parquet with resume support.

Step 3: Fine-tune Embeddings

```
uv run python scripts/finetune_embeddings.py
```

Fine-tunes multilingual-e5-base on labeled data using MultipleNegativesRankingLoss. Produces domain-specific embeddings that separate hazmat from non-hazmat in the embedding space.

Step 4: Tune Hyperparameters

```
uv run python scripts/tune_ensemble.py --n-trials 60
```

Runs Optuna Bayesian optimization to find optimal hyperparameters for XGBoost, LightGBM, RF, ensemble weights, and decision threshold.

Step 5: Train Ensemble

```
make train
```

Trains the final ensemble with optimized params. Registers model and prompts in MLflow. Saves drift baseline.

Step 6: Classify All Items

```
make classify-all
```

Runs the full pipeline: ML ensemble resolves ~94.3% of items, LLM fallback handles ~5.7% of low-confidence items. Human review: 0.07%. Outputs CSV + Parquet.

7. Testing and Quality

7.1 Unit Tests (52 tests)

```
# Run all tests
make test
# or: OMP_NUM_THREADS=1 uv run pytest tests/ -v

# Test coverage:
#   test_api.py       - 15 tests (all endpoints, error handling)
#   test_keywords.py  - 11 tests (hazmat detection, exclusions)
#   test_model.py     - 11 tests (ensemble, save/load, confidence)
#   test_pipeline.py  -  9 tests (ML/LLM routing, thresholds)
#   conftest.py       - shared fixtures and mocks
```

7.2 Linting

```
# Check code quality
make lint

# Auto-fix issues
make lint-fix
```

7.3 Golden Set Validation

```
make validate
```

Runs the model against a curated golden set of items with human-verified labels. Returns pass/fail with exit code gating (non-zero = failed). Quality gates: F1 >= 0.85 and FN rate <= 8%.

7.4 Real-time Testing

```
# Stream random items through the pipeline with detailed metrics
uv run python scripts/realtime_test.py --mode stream --n-items 50

# Single item test
uv run python scripts/realtime_test.py --mode single \
  --title 'Gasolina combustivel 5 litros'
```

7.5 CI/CD (GitHub Actions)

The .github/workflows/ci.yml runs three jobs on every push and PR: lint (ruff check + format), test (pytest with OMP_NUM_THREADS=1), and docker-build (only on push to main).

8. Makefile Reference

Command	Description
make install	Install all dependencies with uv
make serve	Start API + MLflow + Dashboard (all 3 services)
make api	Start FastAPI only (port 8080, hot reload)
make dashboard	Start Gradio dashboard only (port 7860)
make mlflow	Start MLflow UI only (port 5000)
make test	Run 52 unit tests
make lint	Check code quality with ruff
make lint-fix	Auto-fix lint issues
make train	Train ensemble + register in MLflow
make classify-all	Classify 100k items (ML + LLM fallback)
make validate	Validate against golden set (pass/fail)
make promote-staging	Promote latest model to staging
make promote-production	Promote staging to production
make models-list	List all model versions with stage/metrics
make register-prompts	Register LLM prompts in MLflow
make docker-build	Build Docker image
make docker-run	Run with docker-compose

9. Environment Variables

Variable	Required	Description
GEMINI_API_KEY	Yes*	Google AI API key for LLM fallback
MELI_ACCESS_TOKEN	For collection	Mercado Libre API access token
MELI_CLIENT_ID	For collection	Mercado Libre app ID
MELI_CLIENT_SECRET	For collection	Mercado Libre app secret
OMP_NUM_THREADS	No (auto)	OpenMP threads (set to 1 on macOS)
HAZMAT_DEVICE	No (auto)	Force cpu/mps for embeddings
MODELS_DIR	No	Custom models directory path
CANARY_MODELS_DIR	No	Canary model directory for A/B testing
CANARY_PCT	No	Canary traffic percentage (default: 10)

* The API runs ML-only without GEMINI_API_KEY. LLM fallback is disabled gracefully.